Presentation 4 Documentation

• Links to presentation(s) and code(s) on GitHub:

- Video Link
- Dashboard: Power BI Sharing Link
- Call Duration Split Code Link \\ Hierarchical Filtering Code Link

• What did you do?

For this presentation, (1) fixed a small error in my hierarchical menu filter and (2) split up call duration according to time spent in the menus, time spent in queues, and time spent talking to an agent.

1. The hierarchical filter had a mistake for the seniors menu because it didn't have the confirmation menu included. So, I adjusted the hierarchical menus that I built out in the previous presentation in Python and then reloaded it into Power BI with "Seniors Menu" counting only calls that have confirmed they are seniors and therefore reached the "SuburbsOrCityMenu". The filter now accurately reflects calls attributable to seniors and the blank represents the people who choose to abandon the senior menu after confirming, before choosing suburbs or city.
2. I built out a new logic for calculating call duration in stages:

- **Call Duration (Adjusted)** The active length of a call session, measured by pausing and resuming the duration for idle periods between disconnections and callbacks.
- **Menu Time** Time spent by the user navigating menus, derived by subtracting queue time and agent time from the adjusted call duration.
- **Queue Time** Time callers spend waiting in queues. Derived using Krish's logic (see references below).
- **Agent Time** Time spent actively connected to a live agent or chat bot. Derived by looking for time periods with consecutive matching values in the Agent Name column.

Once these changes were made, I loaded the transformed data into Power BI and adjusted my dashboard. My existing visuals (first 2 pages of my dashboard) retained most of the same trends. I performed some analysis to answer your questions from the prior presentation about strange trends, all of which Sophia and I addressed in our video presentation.
Additionally, I built new graphics for the dashboard. The 3rd page breaks down the call duration averages by the 3 stages and compares between seniors and non-seniors. It also shows the total number and proportion of senior callers, as well as the total number and proportion of calls which reach an agent. The entire page can also be filtered by senior/non-senior and agent/non-agent to view more specific information.

• How does it help the project?

By breaking down call durations into menu, queue, and agent stages, we gain actionable insights into how time is distributed across the customer journey. This allows us to identify which menus consistently require longer interaction, pinpoint peak times of day when queues are most demanding, and evaluate how much time is spent with agents. With this clarity, we can make smarter decisions about staffing levels and overall call flow optimization.

• Issues Resolved:

There were a lot of edge cases (calls that didn't match the trends I picked up on in the data) which had incorrect calculations for durations. To find these and resolve the problems, we sorted the Contact Session IDs by longest durations and looked at each one to figure out how to detect where the duration should be paused/resumed. We did this until the trends looked normal and we couldn't find any more cases. This helped me build out my logic to be more comprehensive, which you can see in my code.

• Next steps

I worked mostly on analyzing trends for seniors using these new call durations categories. For my next presentation, I want to use the hierarchical filter to take a deeper look at trends across

these different categories of call duration and see if we can find any insights for other menus. I also want to build out more visuals for these that will be useful to Cynthia when we hand the project off.

• References (Mention if you built up on someone else's work)

- Code import files from Prof. Krishna
- Prof. Krishna's Queue logic - used to develop my logic for queue time

Presentation 3 Documentation

• Links to presentation(s) and code(s) on GitHub:

- Presentation Link (Slides) \\ Presentation Link (GitHub)
- Video Link (Google Drive) \\ Video Link (GitHub)
- Dashboard: Power BI Sharing Link
- Data Prep Code (with call duration fixed) Link \\ Hierarchical Filtering Code Link

• What did you do?

For this presentation, I (1) cleaned up my hierarchical menu filter and (2) fixed the call duration computation according to Cynthia's feedback.

1. I started by manually going through the menus and submenus to see which ones could be consolidated or moved around to make the filter more user friendly. I made the following changes from the previous version:

- Got rid of redundant menu options (e.g., LegalIssues1 led into LegalIssues2 which led into all of the different legal issues, so I got rid of LegalIssues2 since that step took up extra space but did not mean anything)
- Got rid of menus which had very few visits and are irrelevant to our analysis. For example, all of the "GetLoggedIn…" menus are rarely visited and not relevant to LegalAid's data analysis goals, so I scrapped them from the filter.
- Got rid of (some) blank buttons in the filter: blank buttons were crowding the filter and did not have any relevant meaning to the analysis, so I got rid of them. However, some blank buttons remain relevant. For example, if a menu has 2 submenus and blank button option, comparing the blank to the whole menu can show us how many users abandon the menu at this step.
- Grouped some menus together: this mostly applies to queues. If there was a repeated queue name (one SP and one non-SP), I grouped them together under only the queue name since they related to the same legal topic so I could filter for them together.

- Changed menu names to be more readable in the filter (e.g., LegalIssues1 → Legal Issues).

Once these changes were made in the CSV file, I created a python script which outputs the file with all the tiers so that my work can be shared with others. To do this, I built the dataframe from scratch and assigned it to a new CSV file.

After that transformation, I loaded the new hierarchy CSV file into Power BI in place of the old one, still linking it to the compiled CAR data through the Activity_Name column and enabling cross-filtering. From here, I was able to replace the old hierarchy with the new one in the list slicer and filter the visuals on my page.

The other objective I worked towards for this presentation can be seen under “Issues Resolved.”

• How does it help the project?

This cleaner filter allows the user to visualize trends from each queue more clearly, reducing noise from the many submenus. By fixing the call duration calculations to exclude time between disconnections, callbacks, and agent wrap-up periods, the metrics in the dashboard now more accurately represent actual time spent on active calls. Together, these refinements make the Power BI visuals more reliable and user-friendly for identifying performance trends, comparing queue efficiency, and making data-driven decisions about staffing and workflow improvements.

• Issues Resolved:

2. Fixing call duration column:

- Following my Presentation 2, Cynthia noticed that the call duration column in my dataset was overstating total call times because it included periods when the caller had already disconnected or when an agent was wrapping up after the call. To address this, I updated my data processing script to handle different disconnection scenarios.
- The script now detects when a call was paused due to a “DisconnectContact1” event (courtesy callbacks) or a “DisconnectContact” event (manually dialed outbound calls) and accurately resumes timing when the session restarts with a

"LegalServerScreenPop" event. For calls with callbacks, the script calculates duration in multiple segments—before disconnection and after the callback—and sums them for the true total. It also excludes the final timestamp from duration calculations to prevent overcounting, since these include the time that the agent takes to "wrap up" the call.

- After recalculating these adjusted call durations, I merged them back into my aggregated dataset and reloaded the updated data into Power BI, ensuring all dashboard visuals now reflect corrected and more accurate call times.

If you'd like a more detailed layout of the different cases for calls which this new script addresses, see the Appendix at the bottom of this document.

• Next steps

Since I separated the call duration in a new way, I want to dig deeper into this for my next presentation. I want to look at the time periods I have cut out of the call duration column (the time between a user requesting and receiving a callback) and use my hierarchical filter to see if I can uncover any trends by menu for this new metric.

• References (Mention if you built up on someone else's work)

- Code import files from Prof. Krishna
- Visio map with menu triage (from LegalAid)
- I built off of Sophia's Presentation 2 logic combined with the clarifications from Mark Grace about callbacks to develop my new call duration logic.

Presentation 2 Documentation

• Links to presentation(s) and code(s) on GitHub:

- Presentation Link (Slides) \\ Presentation Link (GitHub)
- Dashboard: Power BI Sharing Link
- Data Prep Code Link \\ Hierarchical Filtering Code Link

• What did you do?

For this presentation, I updated my visuals from Presentation 1 on the Time Trends Dashboard and came up with new insights.

I started by moving all of the transformations I previously completed in Power BI over to Python, as requested by Prof. Krishna and Cynthia. I used the import code provided by Prof. Krishna for the CAR data and then made the following transformations in Python:

1. Grouped by Contact Session ID to ensure each call corresponds to a single session and prevent double-counting of calls.
2. Aggregated session-level metrics:
    - Call_Start_Time: timestamp of the first activity in the session
    - Starting_Hour: hour of day when the call started
    - Count: number of distinct activities per session
    - Call_Duration: difference (in minutes) between the first and last activity
    - DayOfWeekNum: numeric representation of the weekday (Monday = 1, Sunday = 7)
    - Call_Start_Date: Just the date (no timestamp) for the first activity of the session

I previously expanded the data in Power BI using the “All Rows” feature for filtering purposes. Python does not allow for this feature, so I instead created a link between the transformed dataset and the original compiled dataset in Power BI via the Contact Session ID column. Then, I set the Cross-Filter direction of that relationship to “Both” to enable filtering. This allows me to

reference details about each call (such as Activity, EP, Queue, Agent, etc.) on an activity level, while still having only one row per call.

I then completed the following actions in Power BI:

3. Created Visualizations:

- Type: Bar Chart
  X-axis:Starting Hour
  Y-axis: Average Calls per Day → To build measure:

```
Avg Calls per Day =
VAR TotalCalls = COUNTROWS('combined_calls_transformed_simple')
VAR TotalDays =
DISTINCTCOUNT('combined_calls_transformed_simple'[Call_Start_Date])
RETURN
DIVIDE(TotalCalls, TotalDays)
```

  Legend: DayofWeekNum
- Type: Line Graph
  X-axis: Starting Hour
  Y-axis: Average of Call Duration per Hour → To build measure:

```
Average Call Duration per Starting Hour =
AVERAGEX(
    VALUES('combined_calls_transformed_simple'[Starting_Hour]),
    CALCULATE(AVERAGE('combined_calls_transformed_simple'[Call_Duration]))
)
```

  Legend: DayofWeekNum

4. Added Hierarchical Filtering:

I used the Visio Map resource to categorize the unique values in the *Activity Name* column of the combined CAR data into a hierarchy by detailing each value with a tier and the tiers that precede it. I then created a dataframe which contained each tier as a separate column and filled in the hierarchy for each activity name, allowing for hierarchical filtering and analysis in Power BI without duplication or blank subfilters. I uploaded this file into Power BI and linked it with the original compiled CAR data via the Activity Name column. Again, I set the Cross-Filter direction of that relationship to “Both” to enable

filtering. I then set up the filtering using a list slicer in Power BI, as seen in the YouTube video provided by Prof. Krishna.

• How does it help the project?

This analysis of peak call time trends with various filters can help the team uncover operational inefficiencies and develop strategies to improve overall intake performance, as seen in my presentation. Based on my analysis, call volumes and durations generally peak in the morning and early in the week across most menus. However, certain categories—such as the **Seniors** menu and its submenus—show significantly longer average call durations, indicating more complex interactions that may require additional training or specialized support. Additionally, language-specific transfers like **StaffDirectorySpanishTransfer** peak later in the day compared to English transfers, highlighting opportunities to better align staffing schedules and expertise with caller behavior patterns.

• Issues Resolved:

- I was having trouble simultaneously grouping the data and retaining all the necessary activity level information in the CAR data. To solve this problem, I linked call-level and activity-level versions of the data in Power BI and enabled cross-filtering.
- I couldn't figure out how to create hierarchical tiers from just one row (Activity Name) in the data. To resolve this issue, I created a separate file with activity tiers and linked it to the CAR data to enable filtering. I then used a list slicer in Power BI and used each tier as a level for the hierarchical filter. I also noticed that a lot of smaller (less frequently visited menus) were populating the top of the filter because they did not belong to other larger menus. To solve this, I remade the activity tier file with a miscellaneous tier so these irrelevant menus could still be accessed without crowding the filter box.

• Next steps

For the next presentation, I plan to:

- Create another hierarchical filter for the “Queue Name” column in the CAR data for my dashboard, using the same process. From there, I will analyze trends in call volume and duration by time and weekday for different Queues and develop targeted recommendations.

• References (Mention if you built up on someone else’s work)

Code import files from Prof. Krishna

Presentation 1 Documentation

• Links to presentation(s) and code(s) on GitHub:

- Presentation Link
    - Note that if the Power BI embedded in the slides fails, static visuals can be found at the bottom of the slide deck
- All coding was done in Power BI: see below // Power BI Sharing Link

• What did you do?

I created visuals for the Time Trends Dashboard. I began by compiling all CAR data into a CSV file using the import code provided by Prof. Krishna. I then loaded the file into Power BI for analysis. I then performed the following transformations on the dataset *within Power BI***:**

I transformed the data using the *Group By* function to ensure each *Contact Session ID* corresponded to only one row—preventing double counting caused by multiple actions per session. I used the *Advanced Group By* option to create multiple aggregations that support deeper analysis and visualization. Specifically, I created three new columns:

- Call Start Time — *Operation:* Minimum of *Activity Start Timestamp*

  Purpose: Shows timestamp of the first row in a session to identify when the call began.
- Starting Hour — *Operation:* Minimum of *Hour*

  Purpose: Records the hour of day when the session started.
- All Rows — *Operation:* All Rows

  Purpose: Retains information from each session in nested tables for detailed reference.

  I also *expanded* the other columns from this one to access nested information.

I then used those columns to create the following columns:

- Count - *Operation*: Row/activity count per call with distinct activity timestamp

  Code: `List.Count(List.Distinct([All Rows][Activity Start Timestamp]))`

  Purpose: See how many different menus/actions a user goes through in each call
- Call Duration - *Operation*: Subtract the minimum timestamp of a call from the maximum

Code: Duration.TotalMinutes(

List.Max([All Rows][Activity Start Timestamp]) -

List.Min([All Rows][Activity Start Timestamp])

)

Purpose: Determine how long each call lasted in minutes

- DayOfWeekNum - *Operation*: Extract weekday from Call Start Time Column as number (Monday = 1 and Sunday = 7)

  Code: Date.DayOfWeek([Call Start Time], Day.Monday) + 1

After these transformations, I created the following visualizations:

- Type: Bar Chart
  X-axis:Starting Hour
  Y-axis: Contact Session
  Legend: DayofWeekNum
- Type: Line Graph
  X-axis: Starting Hour
  Y-axis: Average of Activity Count per Hour → Code for Measure:

```
Average Count per Starting Hour =
AVERAGEX(
   KEEPFILTERS(VALUES('combined_calls'[Starting Hour])),
   CALCULATE(AVERAGE('combined_calls'[Count])))
```

- Type: Line Graph
  X-axis: Starting Hour
  Y-axis: Average of Call Duration per Hour → Code for Measure:

```
Count of Call Duration average per Starting Hour =
AVERAGEX(
   KEEPFILTERS(VALUES('combined_calls'[Starting Hour])),
   CALCULATE(COUNTA('combined_calls'[Call Duration])))
```

*On each visual, you can filter by: Activity Name, Agent Name, EP Name, Flow Name, Queue Name, Termination Reason, and both axes and legend.

• How does it help the project?

This analysis of peak call time trends with various filters can help the team uncover operational inefficiencies and develop strategies to improve overall intake performance, as seen in my presentation. Based on my analysis, changing the messaging for times users are encouraged to call could increase the quality of service for users and allow agents to reach more people overall. Additionally, developing a system for tracking call outcome/success will enable deeper analysis of call effectiveness, help identify when and why shorter calls are less successful, and guide future staffing and training decisions.

• Issues Resolved After Feedback:

- Activities with the exact same timestamp were counted as one and activities with different timestamps were counted as distinct.
- All 3 plots pull from the CAR data to ensure consistency.
- Conclusions about peak call times were adjusted to accurately reflect the dashboard.
- Recommendations were expanded to include all 3 plots and made more precise/actionable.

• Next steps

For the next presentation, I plan to:

- *Dive deeper into the EP/Activity/Flow/Queue filters* to clearly display trends in peak call times by different groups or legal issues
    - Use heatmaps or line charts to highlight patterns across hours and days.
- *Analyze average activities per call and call durations by weekday* in addition to time of day to discover new trends in calling behavior
- *Determine a metric/indicator for call success* which I observe according to time trends

• References (Mention if you built up on someone else's work)

Code import files from Prof. Krishna

Appendix

Adjusted Call Duration Calculation — Case Logic Overview (for Presentation 3):

1. Standard Calls (No Disconnects or Callbacks)
    - Duration = time between the *first* and the *second-to-last* timestamp (last timestamp excluded because it represents agent wrap-up).
2. Single Disconnect
    - If "DisconnectContact1" appears and no "CallbackRetry" is present:
        - Duration pauses at the DisconnectContact1 timestamp.
        - If a "LegalServerScreenPop" occurs afterward, the call resumes there.
        - Total duration = time before disconnect + after callback.
        - If no callback occurs, duration ends at the disconnect time.
3. Multiple Callbacks
    - If both "DisconnectContact1" and one or more "CallbackRetry" events appear:
        - The call pauses before each "CallbackRetry" (at the timestamp preceding it).
        - It resumes at each following "LegalServerScreenPop".
        - All active segments are summed to calculate total duration.
4. Manual Outbound Calls (DisconnectContact)
    - If "DisconnectContact" appears (used for manually dialed outbound calls):
        - The call starts *after* the DisconnectContact timestamp.
        - The call ends at the timestamp preceding the final event (to remove wrap-up).
        - Duration = that adjusted segment only.
5. General Rule for All Cases
    - The last timestamp of every session is excluded, since it represents wrap-up activity rather than live call time.